

eKYC Technical Proposal

Prepared by



For



Table of Contents

Serverless eKYC Architecture	3
Introduction	3
Architecture Overview	4
Layer-by-Layer Architecture	5
1. User Interaction Layer.....	5
2. API Gateway Layer	7
3. Security and Access Control Layer	10
4. Face Liveness Processing Layer	14
5. Document Processing Layer.....	18
6. eKYC Processing Layer.....	23
7. Messaging & Event Layer	27
8. External API Integration Layer	32
9. Storage Layer	39
10. DevOps & Monitoring Layer.....	45
11. DevOps Infrastructure.....	46
End-to-End Process Flow	47
Security Considerations	48
Scalability and Performance	48
Disaster Recovery.....	49
Cost Optimization	49

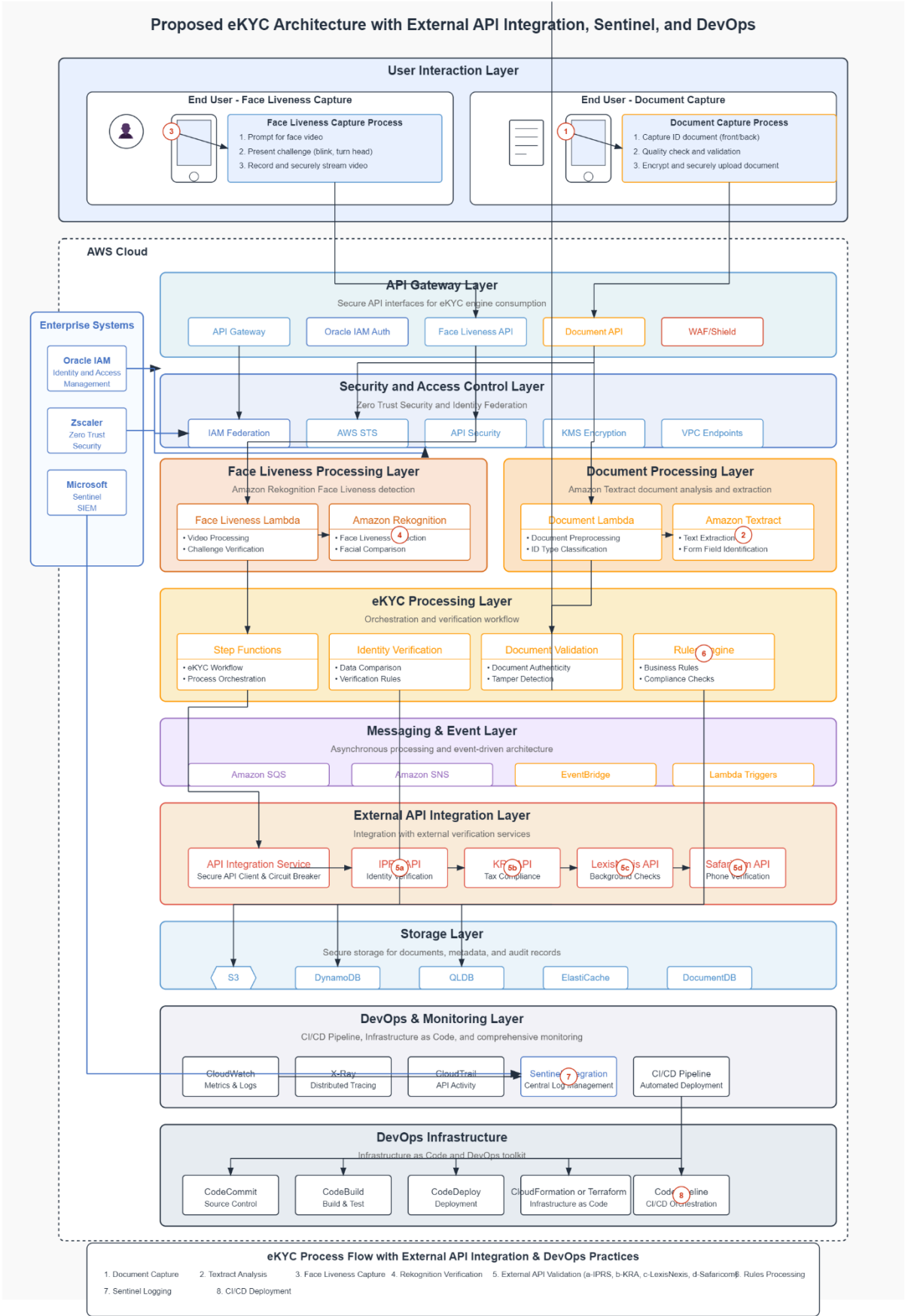
Serverless eKYC Architecture

Introduction

This enhanced architecture builds upon the original eKYC design with AWS best practices and serverless optimizations. The system maintains all existing integration points while improving performance, security, reliability, and cost-effectiveness.

The architecture implements a fully serverless approach using AWS services to create a highly scalable, resilient, and cost-effective eKYC solution that integrates with external verification services, implements comprehensive security controls, and follows DevOps best practices.

Architecture Overview



Layer-by-Layer Architecture

1. User Interaction Layer

Original Components:

- **End User - Face Liveness Capture (3-step process):**
 - Prompts user for face video using device camera
 - Presents challenge sequences (blink, turn head) to verify liveness
 - Records and securely streams video for processing
 - Implemented as responsive web or mobile interfaces
- **End User - Document Capture (3-step process):**
 - Captures ID document using device camera or file upload
 - Performs client-side quality checks and validation
 - Encrypts and securely uploads document
 - Supports various document types (passport, ID card, driver's license)

Enhancements:

- **Multi-device support with responsive design:**
 - Uses AWS Amplify for cross-platform frontend development
 - Implements adaptive UI elements that adjust to different screen sizes
 - Optimizes camera access across iOS, Android, and web browsers
 - Uses CSS media queries and fluid layouts for consistent experience
- **Progressive web app capabilities for offline capture:**
 - Implements service workers for offline functionality
 - Caches essential resources and capture flows
 - Provides queueing mechanism for uploads when connectivity is restored
 - Manages local storage with proper encryption for offline data
- **Client-side validation for immediate feedback:**
 - Uses JavaScript libraries for real-time document edge detection
 - Implements client-side blur/glare detection for document capture
 - Provides facial positioning guidance with real-time feedback
 - Validates input before submission to reduce server-side rejections
- **Device capability detection for optimal capture methods:**
 - Detects camera quality and adjusts capture parameters accordingly

eKYC Proposal

- Adapts to available device sensors (accelerometer, gyroscope)
- Offers alternative flows for devices without camera capabilities
- Implements graceful degradation for older browsers/devices

Integration Points:

- Connects to user devices through web browsers or native applications
- Interfaces with device cameras and file systems for capture
- Communicates with API Gateway for data transmission
- Receives session tokens and challenge parameters from backend
- Implements WebSockets for real-time feedback during capture

Flow:

1. User initiates eKYC process through web or mobile application
 - Application authenticates user session
 - Backend generates unique process ID and session tokens
 - Frontend initializes capture flows
2. System determines optimal capture methods based on device capabilities
 - JavaScript detection of camera quality and browser support
 - Dynamic loading of appropriate capture modules
 - Adaptation of UI elements based on device characteristics
3. User completes face liveness capture (prompt, challenge, recording)
 - System presents random challenge sequence
 - Real-time feedback guides user through the process
 - Video captured in memory with appropriate compression
 - Preliminary client-side liveness checks performed
4. User completes document capture (upload, validation, encryption)
 - Document edges automatically detected and cropped
 - Multiple images captured for highest quality
 - Client-side image enhancement applied
 - Preliminary document type detection performed
5. Captured data is securely transmitted to API Gateway Layer
 - Client-side encryption applied before transmission
 - Chunked uploads for large video files
 - Secure TLS connection established

- Upload progress indicators provided to user

2. API Gateway Layer

Original Components:

- **API Gateway interfaces for eKYC service consumption:**
 - RESTful API endpoints for all eKYC functions
 - Structured API documentation with Swagger/OpenAPI
 - API versioning strategy for backward compatibility
 - Common request/response formats across services
- **Oracle IAM Auth:**
 - Integration with existing Oracle Identity and Access Management
 - Token-based authentication mechanisms
 - Role-based access control for API resources
 - Session management for multi-step processes
- **Face Liveness API:**
 - Endpoints for initiating liveness checks
 - Challenge sequence generation and validation
 - Video stream processing and frame extraction
 - Result callback mechanisms
- **Document API:**
 - Document upload and processing endpoints
 - Support for multipart uploads for large documents
 - Document type classification interfaces
 - Extraction result retrieval
- **WAF/Shield:**
 - Basic protection against common web vulnerabilities
 - DDoS mitigation capabilities
 - IP-based access controls
 - Bot detection mechanisms

Enhancements:

- **API Gateway throttling and caching for performance:**
 - Per-client throttling limits to prevent abuse

- Implementation of burst limits for traffic spikes
 - Response caching for non-sensitive, repetitive calls
 - Cache invalidation strategies based on data changes
 - Stage variables for environment-specific configurations
- **API Gateway usage plans for rate limiting:**
 - Tiered API access based on client needs
 - API key management for client identification
 - Quota management for monthly/daily request limits
 - Throttling limits based on client tier
 - Usage metrics and reporting dashboards
- **AWS WAF with financial services rule sets:**
 - Implementation of OWASP Top 10 protections
 - Financial services specific rule sets
 - Geographic restrictions where appropriate
 - Rate-based rules for brute force prevention
 - Custom rules for business-specific threats
 - Bot control with managed rule sets
- **API Gateway request validation:**
 - JSON Schema validation for all requests
 - Strong request parameter validation
 - Required header validation
 - Content-type enforcement
 - Schema-based validation with detailed error responses
 - Input sanitization to prevent injection attacks

Integration Points:

- **Connects to User Interaction Layer for data ingestion:**
 - Acts as entry point for all client-side requests
 - Provides SDK generation for client applications
 - Handles CORS for web application access
 - Implements WebSocket APIs for real-time communications
- **Integrates with Security and Access Control Layer for authentication:**

- Implements Lambda authorizers for token validation
 - Connects to Oracle IAM for identity verification
 - Enforces access control policies
 - Manages API keys for service-to-service communication
- **Routes to appropriate processing services (Face/Document):**
 - Implements service proxy integration with Lambda
 - Maps request parameters to Lambda event objects
 - Transforms responses from Lambda to API clients
 - Handles error cases with appropriate HTTP status codes
- **Logs all activity to CloudTrail via DevOps layer:**
 - Records all API calls for audit purposes
 - Implements custom access logging
 - Provides execution logs for troubleshooting
 - Captures metrics for monitoring and alerting

Flow:

1. Incoming requests authenticated against Oracle IAM
 - API Gateway receives client request with authentication token
 - Lambda authorizer validates token with Oracle IAM
 - Authorization context generated with user roles and permissions
 - Valid requests proceed, invalid requests rejected with 401/403
2. WAF/Shield protects against threats
 - Request evaluated against WAF rules before reaching API Gateway
 - Malicious requests blocked with appropriate response codes
 - Rate-based protection applied during traffic spikes
 - Geographic restrictions enforced where applicable
3. API Gateway validates request format
 - JSON Schema validation applied to request body
 - Required parameters checked for presence and format
 - Data types validated for all inputs
 - Request rejected with 400 if validation fails
4. Throttling applied based on usage plans

- Request rate checked against client's throttling limits
 - Burst allowances applied for occasional spikes
 - Quotas checked for overall usage limits
 - Requests exceeding limits receive 429 responses
5. Requests routed to appropriate backend services
- API Gateway maps request to appropriate Lambda function
 - Request parameters transformed to Lambda event format
 - Lambda invoked with authorization context
 - Response mapped back to API format for client
 - Cache applied for eligible responses

3. Security and Access Control Layer

Original Components:

- **Zero Trust Security and Identity Federation:**
 - Implementation of zero trust principles (never trust, always verify)
 - Identity-based access controls for all resources
 - Continuous validation of access at each layer
 - Contextual authentication factors (device, location, behaviour)
 - Minimal privilege access model
- **AWS STS (Security Token Service):**
 - Temporary, limited-privilege credentials management
 - Token expiration and rotation mechanisms
 - AssumeRole functionality for cross-account access
 - GetFederationToken for federated identity access
- **IAM Federation:**
 - Integration with Oracle IAM as identity provider
 - SAML 2.0 based federation for enterprise users
 - OAuth/OIDC integration for customer identities
 - Attribute-based access control mappings
 - Role assumption based on federated attributes
- **API Security:**
 - Authorization mechanisms for all API calls

- Mutual TLS for service-to-service communication
- API key rotation and management
- Request signing and validation
- Input validation and sanitization
- **KMS Encryption:**
 - Envelope encryption for all sensitive data
 - Customer managed keys for regulatory compliance
 - Automatic key rotation policies
 - Key usage auditing and monitoring
 - Integration with AWS services for transparent encryption
- **VPC Endpoints:**
 - Private connectivity to AWS services
 - Network-level isolation for sensitive services
 - Endpoint policies for access control
 - Service-specific endpoints for each AWS service
 - No exposure to public internet

Enhancements:

- **AWS PrivateLink for secure service connections:**
 - Private connectivity between VPCs and AWS services
 - Secure connections to third-party services
 - Elimination of data exposure to public internet
 - Network-level isolation with endpoint services
 - Interface VPC endpoints with DNS resolution
 - Custom service endpoints for internal services
- **Lambda authorizers for fine-grained API access control:**
 - Custom authorizers for complex authorization logic
 - Token-based validation with caching
 - Request parameter-based authorization
 - Integration with existing identity systems
 - Detailed authorization context passing to backend
 - Per-route authorization policies

- **AWS Secrets Manager for API keys and credentials:**
 - Centralized storage of sensitive credentials
 - Automatic rotation of secrets on schedule
 - Versioning of secrets for rollback capability
 - Integration with Lambda for secure retrieval
 - Encryption of secrets at rest
 - Fine-grained access control to specific secrets
- **AWS Certificate Manager for TLS management:**
 - Automated TLS certificate provisioning
 - Certificate renewal and deployment
 - Integration with API Gateway and CloudFront
 - Public and private certificate authorities
 - Certificate validation via DNS or email
 - Strong cipher suite enforcement
- **AWS Shield Advanced for DDoS protection:**
 - Enhanced DDoS protection beyond basic Shield
 - 24/7 access to AWS DDoS response team
 - Real-time DDoS attack visibility
 - Intelligent threat detection
 - Cost protection during DDoS incidents
 - Protection for API Gateway, CloudFront, and network load balancers

Integration Points:

- **Secures all API Gateway requests:**
 - Integrates with API Gateway authorizers
 - Provides authentication context to backend services
 - Enforces TLS for all API communications
 - Implements request validation at network edge
 - Protects against common API vulnerabilities
- **Manages identity federation with Oracle IAM:**
 - Acts as bridge between AWS services and Oracle IAM
 - Translates identity assertions to AWS credentials

- Maps external roles to AWS IAM roles
- Manages session duration and token refresh
- Propagates identity context across services
- **Provides encryption services to all processing layers:**
 - Offers KMS APIs for encryption/decryption operations
 - Manages encryption contexts for additional security
 - Provides envelope encryption patterns
 - Supports both symmetric and asymmetric encryption
 - Enables service-specific encryption requirements
- **Controls secure access to AWS resources:**
 - Enforces IAM policies across all services
 - Implements resource-based policies where applicable
 - Manages service control policies for organizational governance
 - Provides VPC endpoint policies for network-level control
 - Enforces least privilege access to all resources

Flow:

1. API requests authenticated via IAM Federation
 - Request arrives with identity federation token
 - Token validated against Oracle IAM
 - User attributes and group memberships extracted
 - Mapping of external identity to AWS IAM role
 - Authorization decisions made based on identity context
2. Temporary credentials issued via AWS STS
 - STS AssumeRoleWithSAML or similar call made
 - Temporary credentials generated with appropriate permissions
 - Session tags applied based on user attributes
 - Credentials limited to required scope and duration
 - Credential context propagated to downstream services
3. KMS manages encryption/decryption of sensitive data
 - KMS keys accessed using temporary credentials
 - Encryption contexts validated for each operation

- Data encrypted/decrypted using appropriate keys
- Key usage logged for audit purposes
- Automatic key rotation applied per policy
- 4. API Security enforces access policies
 - Request evaluated against IAM policies
 - Resource-based policies checked
 - Service control policies applied
 - Permission boundaries enforced
 - Policy evaluation results determined and enforced
- 5. VPC Endpoints ensure traffic doesn't traverse public internet
 - Service requests routed through VPC endpoints
 - Endpoint policies enforce additional access controls
 - Traffic remains within AWS network
 - DNS resolution directs to private endpoints
 - Private IP addressing used throughout

4. Face Liveness Processing Layer

Original Components:

- **Amazon Rekognition Face Liveness detection:**
 - Pre-built ML service for face liveness verification
 - Passive and active liveness detection capabilities
 - Detection of presentation attacks (photos, masks, screens)
 - Confidence scores for liveness determination
 - Integration with other Rekognition APIs
- **Face Liveness Lambda (Video Processing, Challenge Verification):**
 - Dedicated Lambda function for video pre-processing
 - Frame extraction from video streams
 - Temporal sequence analysis for challenge verification
 - Video quality assessment and enhancement
 - Integration with downstream processes
 - Challenge sequence validation logic
- **Amazon Rekognition (Face Liveness Detection, Facial Comparison):**

- Face detection in extracted frames
- Facial attribute analysis
- Facial comparison against reference images
- Similarity score calculation
- Facial landmark detection
- Face pose estimation for challenge verification

Enhancements:

- **Lambda Power Tuning for optimal performance:**
 - Automated testing of different memory/CPU configurations
 - Performance vs. cost optimization
 - Identification of ideal memory settings for video processing
 - Implementation of AWS Lambda Power Tuning tool
 - Periodic re-evaluation based on workload changes
 - Separate configurations for different processing types
- **Provisioned Concurrency for critical functions:**
 - Pre-warmed execution environments for Face Liveness Lambda
 - Elimination of cold start latency
 - Configuration based on traffic patterns
 - Auto-scaling of provisioned concurrency
 - Cost optimization through scheduled concurrency
 - Performance predictability during peak periods
- **Lambda Insights for enhanced monitoring:**
 - Detailed function performance metrics
 - Memory utilization tracking
 - CPU utilization monitoring
 - Network performance analysis
 - Error and exception tracking
 - Initialization time monitoring
 - Correlation with business metrics
- **Step Functions Express Workflows for orchestration:**
 - High-volume, short-duration workflow orchestration

- Parallel processing of multiple verification steps
- Error handling and retry mechanisms
- State management for complex verification flows
- Visual workflow design and monitoring
- Integration with Lambda and Rekognition services
- **Amazon Rekognition Custom Labels for specialized detection:**
 - Custom models for specific verification scenarios
 - Training on domain-specific presentation attacks
 - Enhanced accuracy for particular document types
 - Detection of region-specific ID features
 - Custom confidence thresholds for different use cases
 - Continuous model improvement through feedback loops

Integration Points:

- **Receives video data from API Gateway:**
 - S3-signed URLs for large video uploads
 - Direct Lambda integration for metadata
 - SQS queues for asynchronous processing
 - Event notifications for processing triggers
 - WebSocket connections for real-time feedback
- **Interfaces with Amazon Rekognition services:**
 - Synchronous API calls for immediate responses
 - Asynchronous operations for batch processing
 - Storage of reference faces in Rekognition collections
 - Model selection based on verification requirements
 - Result caching for performance optimization
- **Outputs verification results to eKYC Processing Layer:**
 - Structured result format with confidence scores
 - Detailed metadata for audit purposes
 - Multi-factor verification results
 - Challenge completion confirmation
 - Pre-processing quality metrics

- Face match confidence scores
- **Sends events to Messaging & Event Layer:**
 - Standardized event schema for processing milestones
 - Status updates for asynchronous processing
 - Error notifications with detailed context
 - Completion events with result summaries
 - SQS message publishing for downstream processing
 - EventBridge events for cross-service orchestration

Flow:

1. Face video data received from API Gateway
 - Client uploads video to S3 via presigned URL
 - S3 event notification triggers processing Lambda
 - Processing metadata extracted from request
 - Pre-processing pipeline initialized
 - Challenge parameters retrieved from session context
2. Lambda pre-processes video and extracts frames
 - Video decoded and quality assessed
 - Key frames extracted based on motion detection
 - Frames normalized for lighting and resolution
 - Face detection performed on extracted frames
 - Frame sequence organized based on timestamp
 - Video metadata extracted (fps, duration, resolution)
3. Challenge verification performed on sequence
 - Challenge sequence retrieved from session context
 - Frame analysis for challenge actions (blink, turn head)
 - Temporal sequence verification against expected pattern
 - Confidence score calculated for challenge completion
 - Challenge response time analysed for security
 - Multiple challenges validated independently
4. Rekognition performs liveness detection and facial comparison
 - Processed frames sent to Rekognition

- Liveness detection performed with confidence score
 - Reference image retrieved from secure storage
 - Facial comparison executed against reference
 - Similarity score calculated
 - Anti-spoofing checks performed
5. Results passed to eKYC Processing Layer
- Comprehensive result object created
 - Liveness detection results included
 - Challenge verification results added
 - Facial comparison results incorporated
 - Confidence scores aggregated
 - Result transmitted to eKYC Processing Layer via Step Functions
 - Decision threshold applied based on risk profile
6. Processing events published to Messaging Layer
- Processing completion event published
 - Processing metrics recorded
 - Event contains processing duration and outcome
 - Relevant stakeholders notified based on result
 - Webhook notifications sent if configured
 - Events stored for audit trail

5. Document Processing Layer

Original Components:

- **Amazon Textract document analysis and extraction:**
 - Machine learning-powered document text extraction
 - Form field recognition and extraction
 - Key-value pair identification
 - Table detection and extraction
 - Document structure analysis
 - Support for various document types and languages
- **Document Lambda (Pre-processing, ID Type Classification):**
 - Image pre-processing for quality enhancement

- Document boundary detection and alignment
- Document type classification (passport, ID card, driver's license)
- Quality assessment (blur, glare, lighting)
- Multiple-page document handling
- Document metadata extraction
- **Amazon Textract (Text Extraction, Form Field Identification):**
 - OCR capabilities for document text
 - Form field identification and labelling
 - Field value extraction with confidence scores
 - Key information extraction (name, ID number, expiry date)
 - Table data extraction for complex documents
 - Natural language processing for context understanding

Enhancements:

- **S3 Transfer Acceleration for faster document uploads:**
 - Edge network utilization for optimized upload paths
 - Reduced latency for global users
 - Improved throughput for large document files
 - Progress monitoring during uploads
 - Resumable uploads for interrupted connections
 - Configurable based on document size thresholds
- **SQS FIFO queues for ordered document processing:**
 - Exactly-once processing guarantee
 - Maintains document processing sequence
 - Message grouping by customer/session ID
 - Deduplication to prevent duplicate processing
 - Dead-letter queue for failed processing
 - Visibility timeout management for long-running processes
- **S3 Event Notifications direct to Lambda:**
 - Immediate triggering of processing upon upload completion
 - Event filtering based on object metadata and path
 - Reduced latency compared to polling methods

- Direct invocation pattern for synchronous processing
- Scalable event processing for high upload volumes
- Event batching for efficiency during peak loads
- **Lambda Destinations for asynchronous processing results:**
 - Success/failure routing for asynchronous invocations
 - Direct integration with SQS, SNS, EventBridge
 - Simplified error handling pattern
 - Reduced custom code for result handling
 - Standardized processing result flow
 - Audit trail for all processing attempts
- **Adaptive document classification with machine learning:**
 - Custom trained models for document type recognition
 - Continuous learning from manual corrections
 - Support for region-specific document formats
 - Confidence scoring for classification results
 - Fallback mechanisms for uncertain classifications
 - A/B testing framework for model improvements

Integration Points:

- **Receives document data from API Gateway:**
 - S3 presigned URLs for direct uploads
 - Multipart upload support for large documents
 - Metadata attachment during upload
 - Client-side encryption option
 - Upload progress tracking via WebSockets
 - Request validation before accepting uploads
- **Interfaces with Amazon Textract:**
 - Synchronous API calls for small documents
 - Asynchronous jobs for large, complex documents
 - Job completion notification via SNS
 - Custom extraction templates for specific document types
 - Feature flags for selective extraction capabilities

- Result caching for recently processed documents
- **Outputs extraction results to eKYC Processing Layer:**
 - Standardized data structure for all document types
 - Confidence scores for each extracted field
 - Original and normalized field values
 - Document metadata (type, issue date, expiry)
 - Reference to original document in S3
 - Extraction quality metrics and validation results
- **Sends events to Messaging & Event Layer:**
 - Processing status updates (started, in-progress, completed)
 - Standardized event schema across document types
 - Error events with detailed failure information
 - Success events with processing metrics
 - Workflow transition triggers
 - Webhook notifications for external systems

Flow:

1. Document uploaded via API Gateway to S3
 - Client requests presigned URL for upload
 - Document uploaded directly to S3 using Transfer Acceleration
 - Metadata attached during upload (document type hint, session ID)
 - Client notified of successful upload
 - Upload tagged with customer identifier and timestamp
 - Document stored with server-side encryption
2. S3 Event Notification triggers Document Lambda
 - S3 generates event on successful upload completion
 - Event contains document metadata and location
 - Event filtered based on object path and type
 - Lambda function invoked with event data
 - Concurrency managed based on incoming volume
 - Processing metrics initialized for tracking
3. Lambda pre-processes and classifies document type

- Image downloaded from S3 to Lambda
 - Image quality enhanced (contrast, brightness, sharpness)
 - Document boundaries detected and image cropped
 - Image orientation corrected if needed
 - Document type classified using ML model
 - Processing strategy selected based on document type
 - Quality checks performed (blur detection, resolution check)
4. Textract extracts text and identifies form fields
- Pre-processed image sent to Textract
 - Appropriate Textract API selected based on document type
 - For simple documents: synchronous DetectDocumentText
 - For complex documents: asynchronous AnalyzeDocument
 - Form fields identified and labelled
 - Key-value pairs extracted
 - Tables processed and structured
 - Results normalized and standardized
 - Confidence scores assessed against thresholds
5. Results passed to eKYC Processing Layer
- Extraction results structured in standard format
 - Fields mapped to standard schema regardless of document type
 - Document classification results included
 - Confidence scores for each extracted field provided
 - Document images referenced for further verification
 - Results transmitted via Step Functions state machine
 - Quality metrics included for verification decision
6. Processing events published to Messaging Layer
- Processing completion event published to EventBridge
 - Relevant SQS messages sent for downstream processing
 - Processing metrics recorded in CloudWatch
 - Error details sent to error handling workflow if applicable
 - Notification sent to user if manual review required

- Event correlation IDs maintained for traceability

6. eKYC Processing Layer

Original Components:

- **Orchestration and verification workflow:**
 - End-to-end process management
 - State management across verification steps
 - Error handling and recovery mechanisms
 - SLA monitoring and enforcement
 - Coordination of parallel verification processes
 - Conditional workflow paths based on verification results
- **Step Functions (eKYC Workflow, Process Orchestration):**
 - Visual workflow definition with AWS Step Functions
 - State machine implementation for complex processes
 - Wait states for asynchronous processing
 - Decision points based on verification results
 - Activity polling for human-in-the-loop scenarios
 - Error handling with retry policies
- **Identity Verification (Data Comparison, Verification Rules):**
 - Cross-field validation of extracted data
 - Comparison of extracted data against provided information
 - Logical consistency checks across documents
 - Name matching with fuzzy logic and phonetic algorithms
 - Date validation with format normalization
 - Address standardization and verification
 - Verification rules based on document type and region
- **Document Validation (Document Authenticity, Tamper Detection):**
 - Security feature verification (holograms, watermarks)
 - Document format validation against templates
 - Image manipulation detection
 - Inconsistency detection across document fields
 - Expiration date verification

- Document version validation
- Digital signature verification where applicable
- **Rules Engine (Business Rules, Compliance Checks):**
 - Configurable business rules framework
 - Regulatory compliance rule sets
 - Risk scoring algorithms
 - Decision matrix implementation
 - Rule version management
 - Auditable rule execution
 - Country-specific compliance checks

Enhancements:

- **Step Functions Express Workflows for high-volume processing:**
 - High-throughput, short-duration workflow execution
 - Cost optimization for high-volume verification
 - Sub-second workflow execution for simple cases
 - Reduced latency for end-to-end processing
 - Synchronous execution model when appropriate
 - Execution history for auditing and debugging
- **AWS Lambda Layers for shared code:**
 - Common verification logic shared across functions
 - Centralized libraries for data validation
 - Shared security and encryption utilities
 - Standardized logging and monitoring code
 - Versioned layers for controlled rollout
 - Reduced function package sizes
- **EventBridge Pipes for direct integration points:**
 - Point-to-point integrations with filtering
 - Built-in transformation for event normalization
 - Native enrichment capabilities
 - Simplified integration patterns
 - Reduced custom code for integrations

- Enhanced monitoring and error handling
- **Parallelized verification processes:**
 - Concurrent execution of independent verification steps
 - Optimized processing time for complex workflows
 - Aggregation of parallel process results
 - Failure isolation for individual verification paths
 - Dynamic parallelism based on document complexity
 - Resource optimization during peak loads
- **Machine learning for anomaly detection:**
 - Pattern recognition for fraudulent documents
 - Behavioural analysis for suspicious activities
 - Anomaly scoring based on historical patterns
 - Continuous model training from verification outcomes
 - Integration with fraud detection systems
 - Explainable AI for verification decisions

Integration Points:

- **Receives data from Face Liveness and Document Processing Layers:**
 - Standardized data format for extracted information
 - Structured metadata about processing quality
 - Reference links to original documents and videos
 - Confidence scores from upstream processing
 - Processing metrics for performance analysis
 - Session context preservation
- **Orchestrates verification against external APIs:**
 - Managed integration with third-party verification services
 - Request formatting for each external API
 - Credential management for API authentication
 - Response parsing and normalization
 - Error handling and retry logic
 - Timeout management and circuit breaking
- **Applies business rules and compliance checks:**

- Rule execution against verification data
- Compliance framework integration
- Risk score calculation
- Decision recommendation generation
- Manual review triggering when needed
- Rule execution audit trail
- **Outputs verification results to storage and messaging layers:**
 - Structured verification results for persistence
 - Event publishing for workflow transitions
 - Notification generation for stakeholders
 - Webhook calls to external systems
 - Audit record creation
 - Analytics data preparation

Flow:

1. Step Functions orchestrates end-to-end verification process
 - Workflow initiated with document and face processing results
 - Session context retrieved for correlation
 - Parallel and sequential states defined based on verification requirements
 - Error handling paths configured
 - SLA monitoring enabled
 - Execution history preserved for audit
2. Identity verification compares extracted data with provided information
 - Extracted fields normalized and standardized
 - Reference data retrieved from input or databases
 - Field-by-field comparison with fuzzy matching
 - Cross-document consistency verification
 - Match scores calculated for each field
 - Overall identity confidence score generated
 - Anomaly detection applied to identify suspicious patterns
3. Document validation checks for authenticity and tampering
 - Document template matched based on type and issuing authority

- Security features verified against template
 - Digital validation for electronic documents
 - Tampering detection algorithms applied
 - Consistent formatting validation
 - Expiration validation
 - Document history verification where applicable
4. Rules engine applies business rules and compliance checks
- Business rules executed in predefined sequence
 - Regulatory compliance rules applied based on jurisdiction
 - Risk-based verification depth determined
 - Additional verification triggered when needed
 - Manual review decisions made based on thresholds
 - Final decision calculated from all verification factors
 - Decision justification recorded for audit
5. Results stored in appropriate databases
- Verification results written to DynamoDB
 - Document references stored with results
 - Immutable audit record created in QLDB
 - Decision history preserved for future reference
 - Analytics data aggregated
 - TTL applied to temporary data
6. Verification events published to Messaging Layer
- Decision events published to EventBridge
 - Notification messages sent via SNS
 - Processing metrics sent to CloudWatch
 - Webhook notifications triggered for external systems
 - Workflow state transitions communicated
 - Real-time dashboards updated

7. Messaging & Event Layer

Original Components:

- **Asynchronous processing and event-driven architecture:**

- Decoupled service communication
- Event-based workflow transitions
- Publish-subscribe patterns
- Message-based integration
- Asynchronous processing capabilities
- Event correlation and tracking
- **Amazon SQS:**
 - Managed message queuing service
 - Standard and FIFO queue types
 - Message retention and visibility timeout
 - Dead-letter queue support
 - Delayed message delivery
 - Message batching capabilities
 - Long polling for efficient message retrieval
- **Amazon SNS:**
 - Pub/sub messaging service
 - Topic-based message distribution
 - Multiple subscription protocols (HTTP, email, SQS, Lambda)
 - Message filtering
 - Message archiving
 - Delivery status tracking
 - Large message payloads via S3
- **EventBridge:**
 - Event bus for routing events between services
 - Rule-based event routing
 - Event pattern matching
 - Custom event buses for domain isolation
 - Archive and replay capabilities
 - Schema registry for event structure definition
 - Integration with AWS and third-party services
- **Lambda Triggers:**

- Event-driven function invocation
- Support for various event sources
- Automatic scaling based on event volume
- Synchronous and asynchronous invocation models
- Concurrency management
- Destination configuration for result handling
- Dead-letter queue integration for failed invocations

Enhancements:

- **EventBridge Pipes for point-to-point integrations:**
 - Simplified source-to-target integration
 - Built-in filtering capabilities
 - Native transformation functionality
 - Event enrichment without custom code
 - Enhanced error handling
 - Improved monitoring capabilities
 - Reduced infrastructure overhead
 - End-to-end visibility of message flow
- **SQS dead-letter queues with automated retry policies:**
 - Structured handling of failed message processing
 - Configurable retry attempts before DLQ routing
 - Automated retry with exponential backoff
 - Notification of persistent failures
 - Message age monitoring in DLQ
 - Automated redrive policies
 - Error categorization for failed messages
 - Operational dashboards for DLQ monitoring
- **SNS message filtering for targeted notifications:**
 - Attribute-based message filtering
 - Subscription filter policies
 - Reduced unnecessary message delivery
 - Policy-based message routing

- Dynamic filter policy updates
- Filter evaluation metrics
- Cost optimization through reduced message volume
- Recipient-specific notification content
- **EventBridge Scheduler for delayed processing:**
 - Time-based event generation
 - Scheduled workflow triggering
 - Recurring schedule support
 - One-time future event scheduling
 - Timezone-aware scheduling
 - Flexible scheduling expressions (cron, rate)
 - High precision timing capabilities
 - Integration with existing event targets

Integration Points:

- **Connects all processing layers:**
 - Standardized event schemas across services
 - Common messaging patterns for consistent integration
 - Event correlation through consistent IDs
 - Centralized event bus for system-wide visibility
 - Service mesh-like communication capabilities
 - Event sourcing patterns for state reconstruction
 - Circuit breaking patterns for fault isolation
- **Enables asynchronous communication between services:**
 - Decoupled service interaction
 - Fire-and-forget processing options
 - Request-response patterns via temporary queues
 - Service-to-service communication without direct coupling
 - Load levelling during traffic spikes
 - Back-pressure handling mechanisms
 - Priority queuing for critical messages
- **Triggers Lambda functions based on events:**

- Event-driven function invocation
- Event batching for efficient processing
- Filtering before function invocation
- Concurrent execution based on event volume
- Error handling with destination configuration
- Retry policies for transient failures
- Event source mapping for various event types
- **Interfaces with external notification systems:**
 - Webhook integration for third-party systems
 - Email notifications for user communications
 - SMS delivery for time-sensitive alerts
 - Mobile push notifications
 - Integration with customer communication platforms
 - Audit trail of all external notifications
 - Delivery confirmation tracking

Flow:

1. Processing layers publish events to appropriate channels
 - Standardized event format with metadata
 - Event correlation IDs for traceability
 - Source identification in event payload
 - Event type classification
 - Timestamp and sequence information
 - Security context propagation
 - Custom attributes for specialized routing
2. EventBridge routes events based on pattern matching
 - Events received on appropriate event bus
 - Pattern matching against event attributes
 - Rule evaluation in priority order
 - Multiple targets for matching events
 - Event transformation before delivery
 - Archival for compliance and debugging

- Error handling for delivery failures
- 3. SQS queues manage processing workloads
 - Messages enqueued based on processing requirements
 - FIFO queues for ordered processing
 - Standard queues for high-throughput scenarios
 - Visibility timeout adjusted based on processing needs
 - Dead-letter queue configuration for error handling
 - Long polling for efficient message consumption
 - Batch operations for throughput optimization
- 4. SNS delivers notifications to subscribers
 - Messages published to appropriate topics
 - Filter policies applied at subscription level
 - Delivery to multiple protocols simultaneously
 - Retry logic for failed deliveries
 - Delivery status tracking where supported
 - Message archiving for compliance
 - Large payloads handled via S3 references
- 5. Lambda triggers respond to specific events
 - Functions invoked based on event source mappings
 - Concurrency managed based on workload
 - Event batching for efficiency
 - Success/failure handling via destinations
 - Error handling with retry policies
 - Asynchronous invocation for non-critical processing
 - Synchronous invocation for immediate responses

8. External API Integration Layer

Original Components:

- **Integration with external verification services:**
 - Standardized approach to external API integration
 - Secure communication channels
 - Credential management

- Request/response mapping
- SLA monitoring
- Fault tolerance mechanisms
- API version management
- **API Integration Service (Secure API Client & Circuit Breaker):**
 - Centralized service for external API management
 - Authentication handling for each API
 - Rate limiting and throttling
 - Request logging and monitoring
 - Timeout management
 - Circuit breaker implementation
 - Response caching
 - Error categorization and handling
- **IPRS API (Identity Verification):**
 - Integration with national ID verification system
 - Biographic data validation
 - Identity confirmation services
 - ID number validation
 - Photo verification
 - Address verification
 - Vital records checks
 - Document authenticity verification
- **KRA API (Tax Compliance):**
 - Tax registration verification
 - Tax compliance status checks
 - PIN validation services
 - Business registration verification
 - Tax filing status
 - Tax obligation verification
 - Revenue authority integration
 - Corporate compliance verification

- **LexisNexis API (Background Checks):**
 - Risk screening and verification
 - Sanctions and watchlist checking
 - Politically exposed person screening
 - Adverse media screening
 - Global identity verification
 - Business verification
 - Court records checks
 - Professional license verification
 - Anti-money laundering checks
- **Safaricom API (Phone Verification):**
 - Mobile number verification
 - SIM card ownership verification
 - Phone number age checks
 - SMS verification capabilities
 - Mobile money account verification
 - Number portability checking
 - Active status verification
 - Telecom risk assessment

Enhancements:

- **API Gateway as a service proxy with caching:**
 - Single entry point for all external APIs
 - Consistent interface for diverse APIs
 - Request rate limiting by service
 - Response caching for frequently accessed data
 - API key management for all services
 - Usage plans for external API consumption
 - Request/response logging
 - Cache invalidation strategies
 - API usage analytics
- **Circuit breakers for fault tolerance:**

- Automatic detection of service failures
- Configurable thresholds for failure determination
- Open/closed/half-open state management
- Fallback mechanisms for degraded operations
- Automatic recovery testing
- Manual override capabilities
- Service health monitoring
- Circuit state metrics and dashboards
- Multi-level circuit breaker hierarchy
- **Request/response transformation:**
 - Schema mapping between internal and external formats
 - Data normalization for consistency
 - Protocol adaptation (SOAP to REST, etc.)
 - Character encoding handling
 - Date/time format standardization
 - Currency format conversion
 - Language translation where needed
 - Field mapping for different naming conventions
 - Complex transformation using mapping templates
- **Response validation and error handling:**
 - Schema validation for all responses
 - Business logic validation of response data
 - Comprehensive error categorization
 - Transient vs. permanent error distinction
 - Error response normalization
 - Fallback data sources for critical failures
 - Detailed error logging and monitoring
 - Error correlation with request parameters
 - Error rate alerting and notification
- **Retry mechanisms with exponential backoff:**
 - Automatic retry for transient failures

- Progressive delay increases between attempts
- Configurable maximum retries attempts
- Jitter implementation to prevent thundering herd
- Retry budget per service
- Different strategies based on error type
- Retry attempt logging
- Success rate monitoring after retries
- Circuit breaker integration to prevent unnecessary retries

Integration Points:

- **Connects to eKYC Processing Layer:**
 - Direct integration with Step Functions workflows
 - Event-based triggering of verification processes
 - Lambda function integration for synchronous verification
 - Status notification via callback patterns
 - Result storage in shared data stores
 - Process correlation through session IDs
 - Error propagation mechanisms
 - Verification confidence score standardization
- **Interfaces with external verification services:**
 - Secure HTTPS connections to all external APIs
 - API-specific authentication implementation
 - Request signing where required
 - TLS certificate validation
 - Transport level encryption
 - IP whitelisting where required
 - API version negotiation
 - Rate limiting compliance
 - SLA monitoring
- **Returns verification results to processing workflows:**
 - Standardized result format regardless of source
 - Enriched data with verification metadata

- Confidence scores for verification results
- Verification timestamps and correlation IDs
- Original response preservation for audit
- Error details when verification fails
- Latency information for performance analysis
- Result caching indicators
- **Logs all external API activity:**
 - Comprehensive request/response logging
 - PII redaction in logs
 - Performance metrics collection
 - Error rate monitoring
 - Availability tracking
 - Cost tracking per API
 - Usage patterns analysis
 - Compliance verification
 - Audit trail maintenance

Flow:

1. eKYC Processing Layer initiates external verification
 - Verification request created with required parameters
 - Source data prepared and formatted
 - Request correlation ID generated
 - Verification type determined
 - Required API selected based on verification needs
 - Request prioritization applied
 - Timeout configuration set based on API SLA
2. API Integration Service manages the request
 - Request validation against schema
 - API credentials retrieved from Secrets Manager
 - Request transformation to target API format
 - Rate limiting check before sending
 - Request logging with sensitive data redaction

- Request signing if required
 - Connection pooling for efficiency
 - Timeout configuration applied
3. Circuit breaker prevents cascading failures
- Circuit state checked before request
 - Open circuits prevent request transmission
 - Half-open circuits allow limited testing
 - Failure counting for threshold determination
 - Failure rate calculation over time window
 - Circuit state transitions logged
 - Fallback execution for open circuits
 - Health check scheduling for recovery testing
4. External API provides verification data
- Request transmitted to external service
 - Response timeout monitoring
 - Connection error handling
 - HTTP status code evaluation
 - Response headers processing
 - Response body extraction
 - Rate limit header processing
 - Quota management based on response
5. Response validated and transformed
- Response schema validation
 - Business logic validation of data
 - Response normalization to standard format
 - Field mapping to internal schema
 - Data type conversion where needed
 - Empty/null handling for required fields
 - Special character handling
 - Confidence score calculation
 - Result enrichment with metadata

6. Results returned to eKYC Processing Layer

- Standardized response format regardless of source
- Verification result with confidence score
- Original API response attached for reference
- Error details if verification failed
- Performance metrics included
- Cache status indicated
- Verification metadata attached
- Response delivered via callback or return value
- Event publication for asynchronous flows

9. Storage Layer

Original Components:

- **Secure storage for documents, metadata, and audit records:**
 - Purpose-built storage services for different data types
 - Data lifecycle management
 - Encryption for all stored data
 - Access control mechanisms
 - Backup and recovery capabilities
 - Compliance with regulatory requirements
 - Performance optimization for different access patterns
- **S3 (Simple Storage Service):**
 - Object storage for documents and files
 - Versioning capabilities
 - Lifecycle management policies
 - Server-side encryption
 - Access control with bucket policies and ACLs
 - Cross-region replication
 - Storage class options for cost optimization
 - Event notifications for changes
 - Object lock for WORM (Write Once Read Many) compliance
- **DynamoDB:**

- NoSQL database for structured verification data
 - Key-value and document data models
 - Auto-scaling capabilities
 - Single-digit millisecond performance
 - Secondary indexes for query flexibility
 - Transactions for multiple item operations
 - Continuous backup with point-in-time recovery
 - On-demand and provisioned capacity modes
 - Fine-grained access control
- **QLDB (Quantum Ledger Database):**
 - Immutable, cryptographically verifiable transaction log
 - Document-oriented data model
 - SQL-like query language
 - Automatic journal encryption
 - Cryptographic verification of history
 - Built-in history tracking
 - Streaming capability for integration
 - Independent cryptographic verification
 - Central source of truth for audit data
- **ElastiCache:**
 - In-memory caching service
 - Support for Redis and Memcached engines
 - Sub-millisecond latency
 - High availability with replication
 - Encryption in transit and at rest
 - Automatic failover capabilities
 - Redis AUTH for authentication
 - VPC isolation for security
 - Parameter groups for configuration
- **DocumentDB:**
 - MongoDB-compatible document database

- JSON document storage
- Rich query capabilities
- Horizontal scaling with sharding
- Automatic backup and restore
- Encryption at rest
- Integration with IAM for authentication
- Multiple availability zone deployment
- Storage auto-scaling

Enhancements:

- **S3 Intelligent-Tiering for cost optimization:**
 - Automatic storage class optimization
 - Frequent/infrequent access detection
 - Automatic migration between tiers
 - Monitoring and automation charges optimization
 - Lifecycle rules for archive tiers
 - Object tagging for custom tiering strategies
 - Cost optimization without performance impact
 - No retrieval charges for accessing data
 - Monitoring dashboard for storage distribution
- **DynamoDB Accelerator (DAX) for caching:**
 - In-memory cache for DynamoDB
 - Microsecond read latency
 - Write-through caching
 - Cluster architecture for high availability
 - Seamless DynamoDB API compatibility
 - Reduced read capacity unit consumption
 - Cache invalidation management
 - Item, query, and scan caching
 - Configurable TTL for cached items
- **DynamoDB Global Tables for multi-region support:**
 - Multi-region, multi-active database replication

- Automatic conflict resolution
- Global availability and redundancy
- Read latency reduction for global users
- Disaster recovery capabilities
- Regional compliance for data sovereignty
- Consistent performance across regions
- Seamless failover during regional outages
- Unified management across regions
- **DynamoDB Time-to-Life (TTL) for temporary data:**
 - Automatic item expiration
 - Configurable expiration timestamp attribute
 - Background deletion process
 - No additional cost for TTL processing
 - Reduced storage costs for temporary data
 - Simplified application logic for data clean-up
 - Compliance for data retention requirements
 - Expired item filters for queries
 - TTL stream for deletion notifications
- **QLDB for immutable audit trails with cryptographic verification:**
 - Tamper-evident transaction logging
 - SHA-256 hashing for cryptographic verification
 - Merkle tree implementation for efficient verification
 - Document revision history with timestamps
 - System-assigned sequence numbers
 - Built-in verification tools
 - Export capabilities for external verification
 - Verifiable proof generation
 - Independent auditability of records

Integration Points:

- **Stores data from all processing layers:**
 - Document and image storage from document processing

- Video frame storage from liveness verification
- Structured data from information extraction
- Verification results from identity checking
- External API responses for audit
- Rule execution results for decisions
- Processing metadata for performance analysis
- Event records for workflow tracking
- **Provides retrieval capabilities to verification processes:**
 - Fast document retrieval for comparison
 - Secure access to sensitive verification data
 - Historical verification result access
 - Reference data lookup for validation
 - User profile retrieval for context
 - Document template access for validation
 - Rules and policy retrieval for enforcement
 - Cache-enhanced lookup for frequent operations
- **Maintains audit records for compliance:**
 - Complete audit trail of all verification activities
 - Immutable record of verification decisions
 - User access logging for security audit
 - API call history for troubleshooting
 - Verification process timing for SLA monitoring
 - Rule execution history for decision justification
 - Document access logging for privacy compliance
 - Verification attempt history for fraud detection
- **Enables fast access to frequently used data:**
 - Caching layer for high-frequency lookups
 - In-memory data access for critical paths
 - Performance optimization for common queries
 - Read replicas for high-read workloads
 - Query optimization through indexing

- Data denormalization for access patterns
- Materialized views for complex aggregations
- Result caching for repeated operations

Flow:

1. Documents stored in S3 with appropriate encryption
 - Document uploaded to S3 from processing layer
 - Server-side encryption applied with KMS
 - Object metadata added for classification
 - Lifecycle policy applied based on document type
 - Versioning enabled for change tracking
 - Object lock applied for compliance if needed
 - Access restricted via bucket policies and IAM
 - Object-level logging enabled for auditing
 - Event notifications configured for workflow integration
2. Structured data stored in DynamoDB
 - Verification results written to appropriate tables
 - Data partitioned by customer/session ID
 - Secondary indexes created for query patterns
 - Auto-scaling configured for variable loads
 - TTL set for temporary verification data
 - Transactions used for multi-item consistency
 - Point-in-time backup enabled
 - Strong consistency used for critical reads
 - Global tables enabled for multi-region deployment
3. Verification records stored in QLDB for immutability
 - Decision records written to QLDB ledger
 - All verification steps logged with timestamps
 - Document revisions tracked for changes
 - Cryptographic chaining of all transactions
 - Ledger tables organized by verification type
 - Indexes created for efficient querying

- Export configurations for compliance reporting
- Streaming enabled for event-driven integration
- Verification proofs generated for audit
- 4. Frequently accessed data cached in ElastiCache
 - Reference data loaded into cache
 - Session data cached for ongoing verifications
 - Document metadata cached for quick access
 - Verification status cached for frontend queries
 - Cache invalidation on data updates
 - Tiered caching strategy for different data types
 - Redis data structures used for complex operations
 - Cache clusters sized based on workload
 - Encryption enabled for sensitive data
- 5. Complex document structures stored in DocumentDB
 - Document templates stored as JSON schemas
 - Verification profiles with complex attributes
 - Rule configurations as nested documents
 - Workflow definitions with state transitions
 - Rich querying enabled through indexes
 - Collection-based organization by domain
 - Sharding for high-volume document types
 - Automated backup configured for compliance
 - Read replicas deployed for query-heavy workloads

10. DevOps & Monitoring Layer

Original Components:

- **CI/CD Pipeline, Infrastructure as Code, and comprehensive monitoring:**
 - End-to-end DevOps practices implementation
 - Continuous integration for code quality
 - Continuous delivery for reliable deployments
 - Infrastructure as code for consistency
 - Comprehensive monitoring for visibility

- Alerting for operational issues
- Automated remediation where possible
- Performance optimization based on telemetry
- **CloudWatch (Metrics & Logs):**
 - Centralized metrics collection
 - Custom metrics for business KPIs
 - Log aggregation across services

11. DevOps Infrastructure

Original Components:

- Infrastructure as Code and DevOps toolkit
- CodeCommit (Source Control)
- CodeBuild (Build & Test)
- CodeDeploy (Deployment)
- CloudFormation or Terraform (Infrastructure as Code)
- CodePipeline (CI/CD Orchestration)

Enhancements:

- AWS SAM/CDK for higher-level abstractions
- Infrastructure PR reviews and approval workflows
- Automated compliance checking in pipeline
- Cross-account deployment capabilities
- Blue/green deployment strategies
- Canary deployments for risk mitigation

Integration Points:

- Manages all infrastructure components
- Enables infrastructure as code practices
- Provides consistent deployment across environments
- Ensures infrastructure compliance

Flow:

1. Code stored in CodeCommit repositories
2. CodeBuild compiles and tests code
3. Infrastructure defined in SAM/CDK

4. CodeDeploy manages application deployment
5. CodePipeline orchestrates the entire process

End-to-End Process Flow

1. User Interaction

- User initiates eKYC through web/mobile application
- Completes face liveness and document capture
- Data transmitted to API Gateway

2. API Processing

- API Gateway authenticates and validates requests
- Routes to appropriate processing services
- Applies security controls

3. Parallel Processing

- Face Liveness Processing:
 - Video processed and frames extracted
 - Challenge verification performed
 - Liveness detection and facial comparison executed
- Document Processing:
 - Document pre-processed and classified
 - Text extracted and form fields identified
 - Data structured for verification

4. Verification Orchestration

- Step Functions coordinates verification workflow
- Data comparison performed against provided information
- Document authenticity verified
- Business rules and compliance checks applied

5. External Verification

- Integration with IPRS for identity verification
- KRA API checks for tax compliance
- LexisNexis API for background checks
- Safaricom API for phone verification

6. Result Processing

- Verification results aggregated
- Decision made based on rules engine
- Results stored in appropriate databases
- Notifications sent via messaging layer

7. Audit and Compliance

- All activities logged for audit purposes
- Immutable records stored in QLDB
- Sentinel centralizes log management

8. Continuous Improvement

- Metrics analysed for performance optimization
- User journeys tested with CloudWatch Synthetics
- Infrastructure updates deployed via CI/CD pipeline
- System resilience tested with chaos engineering

Security Considerations

- Zero-trust security model implemented throughout
- Data encrypted at rest and in transit
- Temporary credentials managed via STS
- Fine-grained access control via IAM
- API security with WAF and Shield
- Secure storage with appropriate encryption
- Comprehensive logging and monitoring
- Immutable audit trails for compliance

Scalability and Performance

- Serverless architecture scales automatically
- API Gateway throttling manages load
- Lambda concurrency handles traffic spikes
- SQS decouples processing steps
- Step Functions Express Workflows for high-volume processing
- DynamoDB on-demand capacity for unpredictable workloads

- CloudFront for edge content delivery
- ElastiCache for response time improvement

Disaster Recovery

- Multi-region deployment capability
- DynamoDB Global Tables for data replication
- S3 Cross-Region Replication for documents
- CloudFormation stacks for infrastructure recovery
- Regular backup and restore testing
- Point-in-time recovery for databases
- Automated failover mechanisms

Cost Optimization

- Serverless pay-per-use model
- Lambda memory optimization
- S3 Intelligent-Tiering for storage
- DynamoDB on-demand for variable workloads
- Reserved capacity for predictable components
- Cost allocation tags for visibility
- AWS Budgets for monitoring and alerting

Conclusion

The enhanced serverless eKYC architecture represents a comprehensive solution that balances security, scalability, compliance, and cost optimization. By leveraging AWS's serverless services alongside industry best practices, this architecture delivers a robust platform capable of handling the complex requirements of modern electronic Know Your Customer processes. The solution's layered approach—from user interaction through verification orchestration to secure storage—ensures each component is purpose-built while maintaining seamless integration across the system. The implementation of zero-trust security principles, multi-region resilience, and automated scaling provides a foundation that can adapt to changing regulatory requirements and unpredictable verification volumes. Moreover, the DevOps-focused approach with comprehensive monitoring and CI/CD integration ensures the platform can evolve continuously while maintaining operational excellence. This architecture not only addresses the immediate needs for reliable identity verification but establishes a future-proof framework that can incorporate emerging technologies and adapt to evolving fraud patterns, ultimately delivering a secure, efficient, and cost-effective eKYC solution for years to come.